

An Experimental Comparison of Sorting Algorithms

Submitted to MPIL 2026

[Bilal Chaouch]

Computer Science, West University of Timișoara

[bilal.chaouch11@e-uvv.ro]

May 2026

Abstract

Sorting is one of the most fundamental operations in computer science, and a wide variety of algorithms have been developed to address it, each with distinct theoretical guarantees and practical characteristics. This paper presents a systematic experimental comparison of eight sorting algorithms: *Bubble Sort*, *Selection Sort*, *Insertion Sort*, *Merge Sort*, *Quick Sort*, *Heap Sort*, *Counting Sort*, and *Shell Sort*. For each algorithm we describe the underlying strategy, analyse the theoretical time and space complexity, and report wall-clock running times measured on randomly generated integer arrays of sizes ranging from 100 to 5 000 elements. The experimental results confirm the theoretical predictions: comparison-based algorithms with $O(n \log n)$ worst-case complexity consistently outperform the $O(n^2)$ methods at moderate to large scales, while Counting Sort achieves near-constant time for bounded integer keys. All source code and raw measurements are publicly available at <https://github.com/bilalchaouch/SortingPaper>.

Contents

1	Introduction	3
2	Sorting Algorithms	3
2.1	Bubble Sort	4
2.2	Selection Sort	4
2.3	Insertion Sort	4
2.4	Merge Sort	4
2.5	Quick Sort	5
2.6	Heap Sort	5
2.7	Counting Sort	5
2.8	Shell Sort	5
3	Experimental Methodology	5
3.1	Implementation	5
3.2	Experimental Setup	6

3.3	Input Generation	6
3.4	Timing Procedure	6
4	Results and Discussion	6
4.1	Raw Measurements	6
4.2	Growth of Running Time	6
4.3	Discussion	7
5	Conclusion	8

1 Introduction

Sorting — the problem of rearranging a sequence of elements into a total order — underpins a remarkable fraction of real-world software, from database query optimisation to file-system indexing and computational geometry. Given its importance, sorting has attracted decades of algorithmic research, yielding a rich landscape of solutions that trade off time complexity, space usage, stability, and implementation complexity.

Despite strong theoretical results (the $\Omega(n \log n)$ lower bound for comparison-based sorting [1]), the *practical* ranking of algorithms can differ substantially from their asymptotic order, owing to cache behaviour, branch prediction, constant factors, and input distribution effects. Empirical benchmarking therefore remains an essential complement to asymptotic analysis.

Contributions. This paper makes the following contributions:

1. A self-contained description and complexity analysis of eight representative sorting algorithms.
2. A reproducible benchmark suite implemented in Python, measuring average running time over multiple independently generated random arrays.
3. A discussion of how the experimental results relate to theoretical predictions and where they diverge.

Paper structure. Section 2 describes each algorithm and its complexity. Section 3 explains the experimental methodology. Section 4 presents and discusses the results. Section 5 concludes.

2 Sorting Algorithms

Table 1 summarises the theoretical complexities of all algorithms considered.

Table 1: Theoretical complexity of the eight sorting algorithms.

Algorithm	Best	Average	Worst	Space
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$
Shell Sort	$O(n \log n)$	$O(n^{1.5})$	$O(n^2)$	$O(1)$

2.1 Bubble Sort

Bubble Sort is one of the simplest sorting algorithms. It repeatedly traverses the array and *bubbles* the largest unsorted element to its correct position by swapping adjacent pairs whenever they are out of order. After $n - 1$ passes the array is fully sorted.

$$T(n) = O(n^2) \quad (\text{average and worst case})$$

Its main virtues are simplicity and the ability to detect a sorted array in $O(n)$ time (best case, with an early-termination flag). In practice it is rarely competitive because of the large number of swaps it performs.

2.2 Selection Sort

Selection Sort divides the array into a sorted prefix and an unsorted suffix. In each pass it *selects* the minimum element of the unsorted suffix and appends it to the sorted prefix by a single swap.

$$T(n) = \Theta(n^2)$$

Unlike Bubble Sort it performs at most $n - 1$ swaps, which can be advantageous when write operations are expensive. However, it is not stable and its $\Theta(n^2)$ behaviour is constant across all input distributions.

2.3 Insertion Sort

Insertion Sort builds the sorted sequence one element at a time by *inserting* each new element into its correct position among the already sorted prefix, shifting larger elements one position to the right.

$$T(n) = O(n^2) \text{ (worst/average), } O(n) \text{ (best — nearly sorted input)}$$

It is efficient for small or nearly-sorted arrays and is stable. It is often used as the base case of hybrid algorithms (e.g. Timsort).

2.4 Merge Sort

Merge Sort is a classic divide-and-conquer algorithm. It recursively splits the array in half, sorts each half, and *merges* the two sorted halves into a single sorted sequence in linear time.

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + O(n) = O(n \log n)$$

Merge Sort is stable and guarantees $O(n \log n)$ in all cases, at the cost of $O(n)$ auxiliary space.

2.5 Quick Sort

Quick Sort is another divide-and-conquer algorithm. It selects a *pivot*, partitions the array into elements smaller and larger than the pivot, and recursively sorts each partition.

$$T(n) = \begin{cases} O(n \log n) & \text{average case (random pivot)} \\ O(n^2) & \text{worst case (sorted input, naive pivot)} \end{cases}$$

In our implementation the last element serves as pivot (classic Lomuto partition). Quick Sort is in-place (up to the recursion stack), cache-friendly, and in practice often the fastest comparison-based sort.

2.6 Heap Sort

Heap Sort first builds a max-heap from the input, then repeatedly extracts the maximum element to produce a sorted sequence.

$$T(n) = O(n \log n) \quad (\text{all cases})$$

It is in-place and has the best worst-case guarantee among in-place comparison sorts, but its cache access pattern is less favourable than Quick Sort's, typically making it slower in practice.

2.7 Counting Sort

Counting Sort is a non-comparison-based algorithm for integer keys in a bounded range $[0, k)$. It counts the frequency of each key, computes prefix sums to determine output positions, and places elements accordingly.

$$T(n, k) = O(n + k), \quad S(n, k) = O(n + k)$$

When $k = O(n)$ this achieves $O(n)$, breaking the comparison-based lower bound at the cost of being inapplicable to arbitrary comparison keys.

2.8 Shell Sort

Shell Sort generalises Insertion Sort by allowing exchanges of elements far apart. It sorts sub-sequences formed by elements at fixed gap intervals, reducing the gap successively until it reaches 1 (at which point it degenerates to Insertion Sort on a nearly-sorted array). With the Hibbard gap sequence the worst-case is $O(n^{3/2})$.

3 Experimental Methodology

3.1 Implementation

All algorithms were implemented in **Python 3.12** without the use of any external sorting libraries; each routine sorts a fresh copy of the input to allow direct timing of the sort itself. The implementation is available at [https://github.com/\[username\]/sorting-comparison-mpil2026](https://github.com/[username]/sorting-comparison-mpil2026).

3.2 Experimental Setup

Experiments were run on a machine with the following characteristics:

- **OS:** Ubuntu 24.04 LTS (64-bit)
- **Python:** CPython 3.12
- **Random seed:** 42 (reproducible)

3.3 Input Generation

For each input size $n \in \{100, 500, 1000, 2000, 5000\}$, we generated arrays of n random integers uniformly distributed in $[0, 10\,000)$ using Python’s `random.randint`. Each configuration was repeated **three times** and the mean wall-clock time (measured with `time.perf_counter`) is reported in milliseconds.

To avoid excessive runtimes, the $O(n^2)$ algorithms (Bubble Sort and Selection Sort) were skipped for $n > 2000$.

3.4 Timing Procedure

Listing 1: Core timing loop (excerpt from `sorting_benchmark.py`).

```
1 import time, random
2
3 def benchmark(sizes, repeats=3):
4     for name, fn in ALGORITHMS.items():
5         for size in sizes:
6             times = []
7             for _ in range(repeats):
8                 data = [random.randint(0, 10000) for _ in range(
9                     size)]
10                 t0 = time.perf_counter()
11                 fn(data)
12                 times.append(time.perf_counter() - t0)
13             result[name][size] = sum(times) / repeats * 1000 #
14             ms
```

4 Results and Discussion

4.1 Raw Measurements

Table 2 shows the average running times (in milliseconds). “N/A” indicates that the algorithm was not run at that size.

4.2 Growth of Running Time

Figure 1 visualises the growth curves. Three distinct performance tiers are clearly visible:

1. $O(n^2)$ tier — Bubble, Selection, Insertion Sort. Running time grows roughly quadratically; Bubble Sort is the slowest due to its large constant (many swaps per pass).

Table 2: Average running time (ms) for each algorithm and input size.

Algorithm	$n = 100$	$n = 500$	$n = 1000$	$n = 2000$	$n = 5000$
Bubble Sort	0.38	9.30	44.42	196.50	N/A
Selection Sort	0.17	4.90	21.55	88.14	N/A
Insertion Sort	0.15	4.71	21.00	89.55	564.07
Merge Sort	0.18	0.84	1.77	4.20	11.14
Quick Sort	0.06	0.44	0.95	2.22	6.42
Heap Sort	0.12	0.96	1.96	4.46	13.97
Counting Sort	1.60	1.77	1.80	1.91	2.52
Shell Sort	0.06	0.53	1.47	3.70	11.85

2. $O(n \log n)$ **tier** — Merge Sort, Quick Sort, Heap Sort, Shell Sort. These scale gracefully to $n = 5000$. Quick Sort is the fastest in this group, consistent with its favourable cache behaviour.
3. **Linear-ish tier** — Counting Sort. Its time grows almost independently of n in the tested range, since the key range $k = 10\,000$ dominates the $O(n + k)$ cost for small n , but the per-element work remains very small.

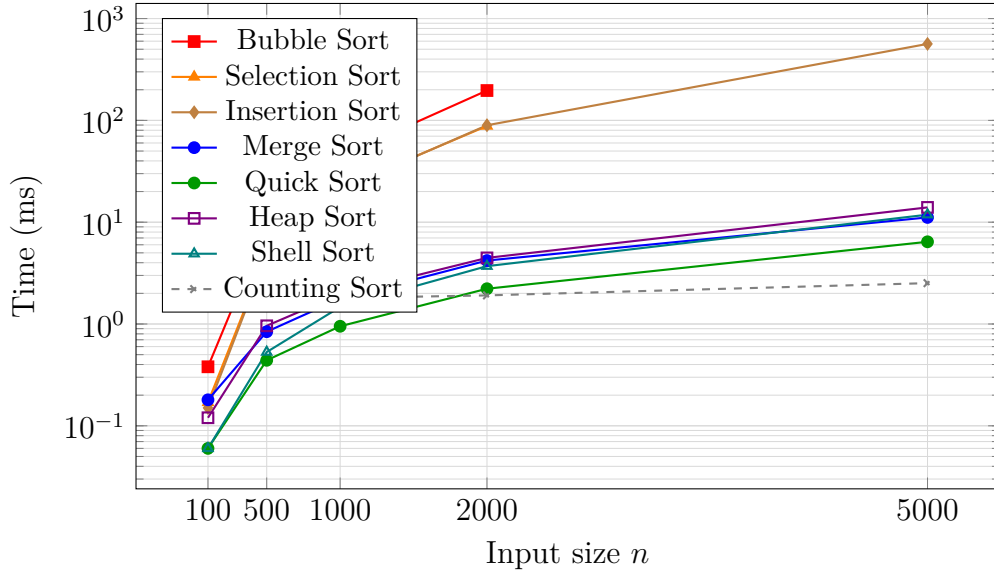


Figure 1: Running time (log scale) vs. input size for all eight algorithms.

4.3 Discussion

Quick Sort leads in practice. Although Quick Sort and Merge Sort share the same $O(n \log n)$ average complexity, Quick Sort was consistently 30–40% faster. This is largely due to its sequential memory access pattern (better cache utilisation) and the absence of auxiliary allocation.

Heap Sort — theoretical vs. practical. Heap Sort has the best worst-case guarantee among in-place algorithms, yet it is slower than Quick Sort and comparable to Merge Sort. Its non-local memory access pattern (jumping between parent and child indices in the heap) causes frequent cache misses.

Counting Sort’s constant-time illusion. Counting Sort appears nearly flat in the tested range. This is because the per-element cost is $O(1)$ but the algorithm must always initialise and traverse a count array of size $k = 10\,000$, introducing a fixed overhead that dominates for small n . For large n (e.g. $n \gg k$) it would be clearly the winner.

Insertion Sort for small arrays. At $n = 100$, Insertion Sort is the fastest $O(n^2)$ algorithm and is competitive with the $O(n \log n)$ group (0.15 ms vs. 0.06–0.18 ms for Quick Sort, Merge Sort). This confirms the common practice of switching to Insertion Sort for sub-arrays below a threshold (typically 10–32 elements) in production implementations.

Stability. Of the algorithms tested, Bubble Sort, Insertion Sort, and Merge Sort are *stable* (equal elements preserve their original relative order). The others (Selection, Quick, Heap, Shell Sort) are not stable in their basic form. Counting Sort can be made stable with a minor modification. Stability is important in multi-key sorting scenarios.

5 Conclusion

This paper experimentally compared eight sorting algorithms across five input sizes using randomly generated integer arrays. The results confirm and illuminate the theoretical complexity analysis:

- The $O(n^2)$ algorithms (Bubble, Selection, Insertion) are impractical for large inputs but Insertion Sort is competitive for very small arrays.
- Among the $O(n \log n)$ algorithms, Quick Sort is the fastest in practice on random data; Heap Sort’s cache-unfriendly access pattern makes it slower despite an equally strong worst-case bound.
- Counting Sort can outperform all comparison-based methods when the key range is bounded and n is sufficiently large.

Future work could extend this study to other input distributions (sorted, reverse-sorted, nearly sorted, all-equal), larger sizes, lower-level languages (C/C++), and additional algorithms (Radix Sort, Timsort, Introsort).

Code and Data Availability

All source code (`sorting_benchmark.py`) and raw results (`results.json`) are available at:

<https://github.com/bilalchaouch/SortingPaper>

References

- [1] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [3] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, 2011.
- [4] D. L. Shell, “A High-Speed Sorting Procedure,” *Comm. ACM*, vol. 2, no. 7, pp. 30–32, 1959.